

# Relatório 1º projecto ASA 2025/2026

**Grupo:** AL054

**Alunos:** Duarte Graça (113395) e Carlota Vicente (115090)

## Descrição do Problema e da Solução

O problema consiste em remover todos os aminoácidos de uma cadeia na ordem que maximize a energia libertada, determinada pelas afinidades entre vizinhos e pelas energias potenciais. Implementámos um programa em C++ que recebe a cadeia e devolve a energia máxima libertada e a ordem ótima de remoção. O cálculo é feito através de um algoritmo que avalia todas as subsequências, identificando o melhor aminoácido a remover por último em cada intervalo, armazenando posições e energias em matrizes usadas para reconstruir a solução ótima.

## Análise Teórica

Assumindo  $n$  como o tamanho da cadeia de aminoácidos:

- Leitura dos dados de entrada: leitura do cin, tornando o input num vetor de aminoácidos. Ou seja,  $O(n)$ .
- Cálculo da afinidade: acesso direto a uma tabela, ou seja,  $O(1)$ .
- Cálculo da energia libertada: aplicação direta fórmula do enunciado.  $O(1)$ .
- Processamento da energia máxima possível: criam-se duas matrizes (uma unsigned long long e outra int) para armazenar, para cada intervalo  $i-j$ , a energia máxima libertada e o último aminoácido removido. Três loops aninhados percorrem todas as subcadeias: os dois primeiros geram todos os intervalos  $i-j$  e o terceiro testa todos os  $k$  possíveis como último aminoácido removido. A energia máxima para a cadeia completa fica em  $(1, n)$ . A complexidade deste processo é  $O(n^3)$ .
- Construção da sequência por ordem de remoção: usando a matriz de remoções, acede-se diretamente ao último aminoácido a remover em cada intervalo. A função é recursiva: após identificar esse aminoácido, chama-se para as subcadeias à esquerda e à direita. Como todos os acessos são diretos a complexidade é  $O(n)$ .
- Loop de print: imprime a energia libertada,  $O(1)$ . Itera o vetor de inteiros que indicam os aminoácidos a ser removidos por ordem,  $O(n)$ .
- Complexidade global da solução é de  $O(n) + O(1) + O(1) + O(n^3) + O(n) + O(n)$ , que se pode reduzir à maior potência  $\approx O(n^3)$ .

Função eMax(vetor<aminoácidos> cadeia)

```
    criar matriz dp[n+2][n+2] de unsigned long long
    criar matriz optK[n+2][n+2] de inteiro
    criar vetor resultado
    for(int i =n; i>=1; i--)
        for(int j =1; j<=n; j++)
            if (i>j) passar ao próximo j endIf
            maxi =0; bestK=-1;
            for(int k=i; k <=j; k++)
                e = eLib(cadeia[k], cadeia[i-1],
                    cadeia[j+1]) + dp[i][k-1] + dp[k+1][j];
```

# Relatório 1º projecto ASA 2025/2026

Grupo: AL054

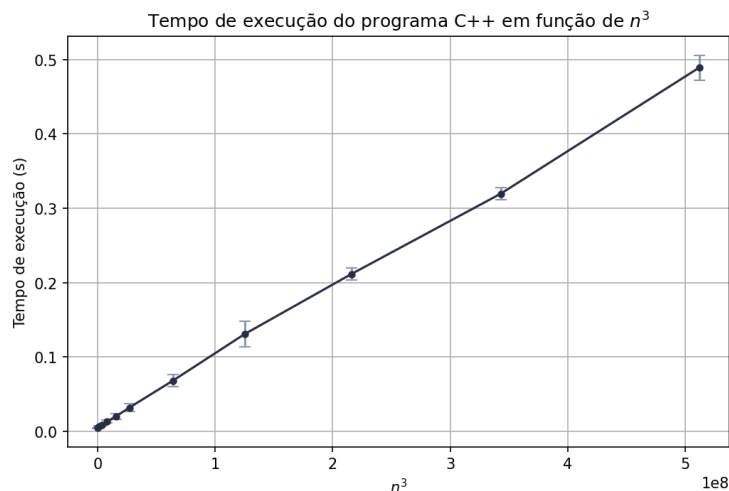
Alunos: Duarte Graça (113395) e Carlota Vicente (115090)

```
        if(e>=maxi)
            maxi =e; bestK = k;
        endIf
    endFor
    dp[i][j] = maxi; optK[i][j] = bestK;
endFor
endFor
seq(1, n, optK, res); return {dp[1][n], res};
endFunction
Função seq(i, j, matriz optK, vetor resultado){
    if (i>j) then return; endif
    k = optK[i][j];
    seq(i, k-1, optK, res); seq(k+1, j, optK, res);
    adiciona k ao final de res
endFunction
```

## Avaliação Experimental dos Resultados

Testámos o nosso código com 11 inputs de energia potencial máxima de 50 e de  $n$  entre 50 e 800. Utilizamos uma *seed* fixa.

O eixo dos XX representa a complexidade teórica ( $n^3$ ) e o eixo dos YY o tempo de execução em segundos. É possível observar que o tempo de execução cresce linearmente com o crescimento teórico esperado, confirmando que, de facto, a solução tem complexidade  $\approx O(n^3)$ .



## Referências:

GeeksforGeeks. "Dynamic Programming or DP." GeeksforGeeks, 9 Feb. 2024, [www.geeksforgeeks.org/competitive-programming/dynamic-programming/](https://www.geeksforgeeks.org/competitive-programming/dynamic-programming/). Accessed 3 Dec. 2025.

W3Schools. "C++ Tutorial." W3schools.com, 2019, [www.w3schools.com/cpp/](https://www.w3schools.com/cpp/). Accessed 3 Dec. 2025.